SIGNALCAST: HOURLY TRAFFIC FORECASTING FOR ADAPTIVE SIGNAL TIMING

A PROJECT REPORT

AD23532 – PRINCIPLES OF DATA SCIENCE

Submitted by

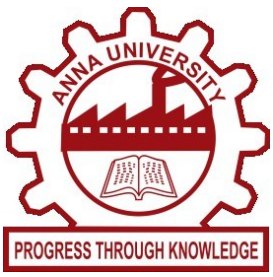
B SUNETRA 241801282

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE





RAJALAKSHMI ENGINEERING COLLEGE
(AUTONOMOUS)

CHENNAI – 602 105.

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this Report titled "**SIGNALCAST: HOURLY TRAFFIC FORECASTING FOR ADAPTIVE SIGNAL TIMING**" is the Bonafide work of "**B SUNETRA (241801282)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Prof. Nirmala Anandhi,

Professor,

Department of Artificial Intelligence and Data science,

Rajalakshmi Engineering College,

Thandalam – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

Initially I thank the Almighty for being with us through every walk of my life and showering his blessings through the endeavor to put forth this report.

My sincere thanks to our Chairman **Mr. S. MEGANATHAN, M.E., F.I.E.,** and our Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, M.E., Ph.D.,** for providing me with the requisite infrastructure and sincere endeavoring educating me in their premier institution.

My sincere thanks to **Dr. S.N. MURUGESAN M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time.

I express my sincere thanks to **Dr. J M GNANASEKAR M.E., Ph.D.,** Head of the Department of Artificial Intelligence and Data Science for his guidance and encouragement throughout the project work.

I convey my sincere and deepest gratitude to our internal guide, **Prof. NIRMALA ANANDHI,** Professor, Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College for valuable guidance throughout the course of the project.

Finally, I express my gratitude to my parents and classmates for their moral support and valuable suggestions during the course of the project.

B SUNETRA

(241801282)

ABSTRACT

SignalCast: Hourly Traffic Forecasting for Adaptive Signal Timing is a Principles of Data Science project that forecasts traffic demand and uses the forecast to set signal green time. Fixed-time signals give every approach the same green time regardless of demand, which wastes green time and increases waiting. The project uses the public Metro Interstate Traffic Volume dataset, cleans it into 12,689 continuous hourly records, and builds leakage-free lag, rolling and calendar features. A seasonal ARIMA model, Random Forest and XGBoost forecast the next hour and are compared with last-value and seasonal-naive baselines under a chronological split, five rolling-origin folds, a feature ablation and an error analysis. Random Forest achieved an RMSE of 200.0 vehicles per hour, an R^2 of 0.989 and a MAPE of 5.1%, which is 74.9% lower RMSE than the last-value baseline, and XGBoost was almost identical and about twelve times faster to train. Mapping the forecast to a 10–60 second green time reduced the mean timing error from 13.4 seconds for a fixed plan to 1.0 second. A Streamlit dashboard presents the results interactively.

Keywords: Traffic Forecasting, Time Series Analysis, Random Forest, XGBoost, Adaptive Signal Timing.

INDEX

CHAPTER NO.	TITLE	PG. NO.
	ABSTRACT	4
	KEYWORDS	4
1	INTRODUCTION	8
1.1	Background	8
1.2	Motivation	9
1.3	Objectives	9
1.4	Problem Statement	10
1.5	Scope of the Project	10
2	LITERATURE SURVEY	11
3	SYSTEM ARCHITECTURE	14
4	SYSTEM ANALYSIS	17
4.1	Existing System	17
4.2	Proposed System	17
4.3	System Requirements	18
5	MODULES DESCRIPTION	19
5.1	Data Collection and Understanding Module	19
5.2	Data Preprocessing Module	20
5.3	Feature Engineering Module	21

5.4	Forecasting Models Module	22
5.5	Evaluation and Analysis Module	23
5.6	Signal Timing and Dashboard Module	23
6	IMPLEMENTATION	24
7	RESULTS AND DISCUSSION	26
8	FUTURE ENHANCEMENTS	35
9	REFERENCES	36

LIST OF FIGURES

FIGURE NO.	TITLE	PG. NO.
3.1	Architecture Diagram	15
5.1	Exploratory Analysis of the Hourly Traffic Volume	20
7.1	Actual and Forecast Traffic Volume	27
7.2	Random Forest Forecast against Actual Volume	27
7.3	Top Features of Random Forest and XGBoost	29
7.4	Random Forest Error by Hour and Error Distribution	30
7.5	Forecast-Based and Fixed-Time Green Time	31
7.6	Dashboard, Forecast Tab	32
7.7	Dashboard, Signal Timing Tab	33
7.8	Dashboard, Insights Tab	33

CHAPTER 1

INTRODUCTION

Road transport is the backbone of urban life in India, and the rapid growth in the number of vehicles has pushed many city junctions far beyond the capacity they were designed for. Most signalised junctions still operate on fixed-time plans in which every approach receives a pre-set green interval whatever the number of vehicles actually waiting. As a result, empty lanes continue to receive green time while crowded lanes build long queues, which increases waiting time, fuel consumption and emissions, and delays emergency vehicles.

The SignalCast project applies the Principles of Data Science to this problem. Traffic volume follows strong daily and weekly rhythms, and these patterns can be learned from historical data. The project collects and cleans an hourly traffic dataset, explores its structure, engineers features from past values, trains time series and machine learning models to forecast the next hour of traffic, evaluates them carefully, and turns the forecast into a signal green time that is shown in an interactive dashboard.

1.1 Background

Traffic signal control has evolved from fixed-time plans, which repeat a cycle tuned offline for an average day, through detector-actuated controllers that react at a single junction, to adaptive systems such as SCATS and SCOOT that adjust timings from live detector data. Most of these systems react to the traffic that has already arrived. A forecast of the traffic that is about to arrive allows timing to be set in advance. Short-term traffic forecasting has been studied with seasonal time series models such as SARIMA and, more recently,

with machine learning models such as Random Forest and XGBoost and with deep networks. Data science supplies the full workflow for this task: data understanding, cleaning, exploration, feature construction, modelling, validation and communication of results.

1.2 Motivation

Traffic congestion wastes time and fuel every day, and its effect is most serious when an ambulance or a bus is delayed in the queue. Fixed-time signals cannot follow the morning and evening peaks, the drop at night or the differences between weekdays and weekends. Forecasting models that are accurate, fast to train and easy to explain would let traffic engineers set green time in proportion to expected demand. The motivation for this project is to find out, with a careful and reproducible comparison on a public dataset, which simple models forecast hourly traffic well enough to support such timing, and what inputs they really need.

1.3 Objectives

The primary objective of this project is to forecast hourly traffic volume and use the forecast to set signal green time. The specific objectives are to clean and prepare a public hourly traffic dataset without inventing data across large gaps; to explore the daily, weekly, weather and holiday patterns in the data; to build features from past values only, so that the forecasts could be used in practice; to compare a seasonal ARIMA model, Random Forest and XGBoost with last-value and seasonal-naive baselines; to test the stability of the result with rolling-origin evaluation, and to understand it with a feature ablation and an error analysis; to convert the forecast into a green time and compare it with

a fixed-time plan; and to present the results in an interactive Streamlit dashboard.

1.4 Problem Statement

Fixed-time traffic signals allocate green time without considering the traffic that is about to arrive, which leads to unnecessary waiting and inefficient use of signal time. Adaptive systems that react only to current detector readings respond after the traffic has already built up. Forecasting studies, in turn, often use different datasets and metrics, rarely compare against strong seasonal baselines, and rarely show which inputs the models need or where they fail. Therefore, there is a need for a reproducible data science pipeline that forecasts hourly traffic volume from historical data, is evaluated carefully against strong baselines, explains what the models rely on and where they make errors, and shows how the forecast can set signal green time.

1.5 Scope of the Project

The scope of this project covers the forecasting of hourly traffic volume on the Metro Interstate Traffic Volume dataset from the UCI Machine Learning Repository and the derivation of a green time from the forecast. The forecasts are made one hour ahead for a single highway location, using a chronological 80/20 split and five rolling-origin folds. The green time is derived from the forecast by a simple rule whose limits are design choices, and it is evaluated against the timing obtained from the actual traffic, so the study measures forecast accuracy in seconds and not the waiting time of vehicles. The project does not include a traffic simulation, pedestrian phases, emergency vehicles or direct control of real signal controllers, which are listed as future work.

CHAPTER 2

LITERATURE SURVEY

Recent advances in Artificial Intelligence (AI), reinforcement learning and data science have created new opportunities for adaptive traffic signal control. Traditional signals rely on fixed-time plans or simple detector logic, which cannot respond properly to changing demand and make little use of historical traffic patterns. The studies below were all published in 2025.

In 2025, researchers proposed “Deep Reinforcement Learning-Based Traffic Signal Control for Urban Congestion” using deep Q-network models at a four-way intersection, modelled as a Markov decision process [1]. The learned controllers reduced average waiting time, queue length and emergency-vehicle delay compared with conventional methods, but the work covered only a single intersection.

“Adaptive Traffic Signal Control based on Multi-Agent Reinforcement Learning. Case Study on a simulated real-world corridor” used multi-agent proximal policy optimization on a simulated seven-intersection corridor [2]. It improved travel time by 2 percent and 24 percent in the primary and secondary coordination directions over actuated-coordinated control, but it relies on trained policies and uses no historical forecasting.

“PairUpLight: A Multi-agent Reinforcement Learning Approach for Coordinated Multi-intersection Traffic Signal Control” lets each intersection learn a small message to exchange with its neighbours instead of using hand-crafted features [3]. It showed good robustness across traffic flow patterns, but it does not use historical forecasting.

“A Constrained Multi-Agent Reinforcement Learning Approach to Autonomous Traffic Signal Control” added GreenTime, GreenSkip and PhaseSkip constraints so that learned policies obey real signal rules [4]. It gained 12.60, 10.29 and 13.10 percent in average reward over MAPPO, IPPO and QTRAN, and its fairness idea matters for any timing rule that must not starve low-traffic lanes.

“Multi-agent Reinforcement Learning for Traffic Signal Control: Comparison with Centralized, Decentralized and Hybrid Approaches” combined a sequence-to-sequence predictor trained on historical data with a value-decomposition network so that control acts on predicted traffic [5]. It outperformed baselines on the Hangzhou16 network and matched them on Jinan12, which supports the use of historical patterns, the approach of this project.

Closest to this project is “Forecasting traffic flow by vehicle category on a major highway impacted by road maintenance works” [6]. It compared Prophet, harmonic regression, seasonal autoregression and SARIMA against a seasonal naive baseline and found SARIMA the best, with holidays and road works as influential factors. This supports SARIMA and a seasonal baseline as reference models, but the study did not include tree-based ensembles.

A 2025 hybrid forecasting model feeds an STL decomposition into an LSTM for the trend, an ARIMA model for seasonality and XGBoost for the residual [7]. On 998 records from a New York intersection it outperformed each standalone model, which suggests that combining time series and boosted models is promising.

On the data side, “CityPulse: Real-Time Traffic Data Analytics and Congestion Prediction” processed 11 million simulated traffic records with Apache Kafka and Spark Structured Streaming and classified congestion as low, medium or high [8]. It stops at monitoring and does not set signal timings.

Based on these studies, learned controllers reduce waiting and travel time and historical forecasting can make control proactive. However, signal-control studies rarely benchmark simple forecasting models on public data, forecasting studies rarely connect the forecast to signal timing, and few studies show which inputs the models need or where they fail. The SignalCast project addresses this gap with a reproducible pipeline on a public dataset, a comparison of SARIMA, Random Forest and XGBoost against strong baselines, rolling-origin testing, a feature ablation, an error analysis and a rule that turns the forecast into green time.

CHAPTER 3

ARCHITECTURE OF THE SIGNALCAST SYSTEM

The architecture of the SignalCast system is a linear data science pipeline with eight stages. The traffic records from the UCI repository are cleaned and placed on a regular hourly grid, explored, and converted into 29 features built from past values only. Baseline, SARIMA, Random Forest and XGBoost models forecast the next hour of traffic volume, and the models are evaluated with a chronological split, five rolling-origin folds, a feature ablation and an error analysis. The best forecast is converted into a green time by a simple rule and compared with a fixed-time plan, and all results are presented in a Streamlit dashboard. Figure 3.1 shows the complete architecture.

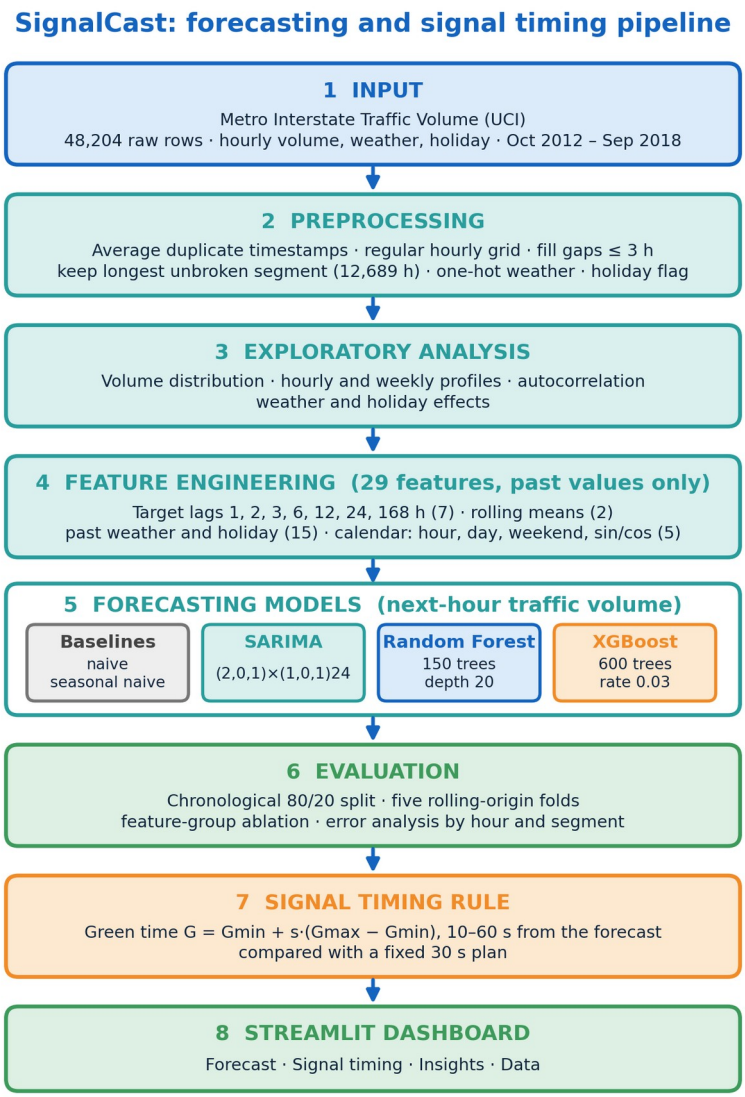


Figure 3.1: Architecture Diagram

The input stage uses the Metro Interstate Traffic Volume dataset [9], with 48,204 hourly rows from October 2012 to September 2018 that combine traffic volume with temperature, rain, snow, cloud cover, weather type and holiday information. The preprocessing stage averages duplicate timestamps, places the series on a regular hourly grid, fills gaps of up to three hours and keeps the longest unbroken segment, which gives 12,689 hourly records from April 2017 to September 2018. The exploratory stage studies the volume

distribution, the hourly and weekly profiles, the autocorrelation and the weather and holiday effects, and the feature stage builds seven target lags, two rolling means, 15 past weather and holiday values and five calendar features.

The modelling stage trains the forecasting models. Two baselines, the last value and the value 24 hours earlier, provide reference points. A seasonal ARIMA model [14] represents classical time series analysis, and Random Forest [12] and XGBoost [11] represent tree-based machine learning. The evaluation stage measures accuracy on a held-out test period and on five rolling-origin folds, removes feature groups one at a time to see what the models need, and breaks the errors down by hour and day type.

The signal timing stage applies a linear rule that maps the forecast volume to a green time between 10 and 60 seconds and compares it with a fixed 30-second plan. The dashboard stage, built with Streamlit [15], lets the user choose the model, the window of the test period and the timing limits, and shows the forecasts, the timing and the diagnostics.

CHAPTER 4

SYSTEM ANALYSIS

4.1 Existing System

In most cities, traffic signals still run on fixed-time plans in which the green interval of every approach is set in advance for an average day and repeated cycle after cycle. Such plans cannot respond to a sudden build-up of vehicles or to the change in demand between peak and off-peak hours, so empty approaches keep receiving green time while crowded approaches accumulate long queues. Detector-actuated controllers improve on this by extending green when vehicles are present, but they react only to the traffic at their own junction, and adaptive systems such as SCATS and SCOOT respond to the traffic that has already arrived. Where traffic is forecast, simple rules such as repeating the last value or the value of the same hour yesterday are common, but these fail when conditions differ from the previous period. Studies of forecasting models also often use different datasets and metrics and rarely explain what the models rely on. This highlights the need for a reproducible, accurate and explainable forecasting approach that can drive signal timing.

4.2 Proposed System

The proposed system is a data science pipeline that forecasts hourly traffic volume and sets green time from the forecast. Public traffic records are cleaned into a continuous hourly series, explored, and converted into features that use only past values. A seasonal ARIMA model, Random Forest and XGBoost forecast the next hour and are compared with last-value and seasonal-naive baselines using a chronological split and five rolling-origin folds. A feature ablation shows which inputs the models need and an error

analysis shows when they fail. A linear rule converts the forecast into a green time that is compared with a fixed-time plan, and a Streamlit dashboard presents the results. The proposed system forecasts demand well, is fast to train, and is explainable because its decisions rest on a small set of understandable inputs.

4.3 System Requirements

The system requires modest hardware and freely available software. The hardware requirement is a computer with a multi-core processor, at least 8 GB of memory and an internet connection for Google Colab; no GPU is needed. The software requirements are Python 3, pandas and NumPy for data handling, scikit-learn [10] for Random Forest and evaluation, XGBoost [11] for gradient boosting, statsmodels [13] for SARIMA, matplotlib for figures, and Streamlit [15] with Plotly for the dashboard. Functionally, the system must load and clean the dataset, build features from past values, train and evaluate the models, convert forecasts to green time and display the results. Non-functional requirements include reproducibility through a fixed random seed, speed of training, explainability of the models, and ease of use through a simple dashboard.

CHAPTER 5

MODULES DESCRIPTION

5.1 Data Collection and Understanding Module

This module collects the data and studies its structure. The Metro Interstate Traffic Volume dataset [9] is a public dataset from the UCI Machine Learning Repository that records hourly westbound traffic volume on a US interstate highway with temperature, rain, snow, cloud cover, weather type and holiday information. The raw file has 48,204 rows from October 2012 to September 2018, with duplicate timestamps and large gaps. Table 5.1 summarises the data used. The exploratory analysis in Figure 5.1 shows that the volume distribution is bimodal, with a night-time mode near 400 vehicles per hour and a daytime mode near 4,800. Weekdays average 3,613 vehicles per hour and weekends 2,712, about 25% less. The weekday profile has a morning peak of about 6,000 at 07:00 and an evening peak of 6,342 at 16:00, while the weekend profile is a broad plateau that peaks at 4,522 at 12:00. The autocorrelation is 0.914 at lag 1, 0.866 at lag 24 and 0.937 at lag 168, so the same hour one week earlier is a better guide than the same hour one day earlier. Mean volume differs by weather type, but these differences are confounded with the time of day, and the 15 holiday hours in the segment are too few for a reliable holiday estimate.

Table 5.1: Dataset summary

Item	Value
Raw rows / period	48,204 / Oct 2012 – Sep 2018
Segment used	13 Apr 2017 – 30 Sep 2018
Hourly records after cleaning	12,689
Target (vehicles per hour)	mean 3,353; std 1,977
Target range	151 – 7,213
Features	29
Training / test hours	10,151 / 2,538

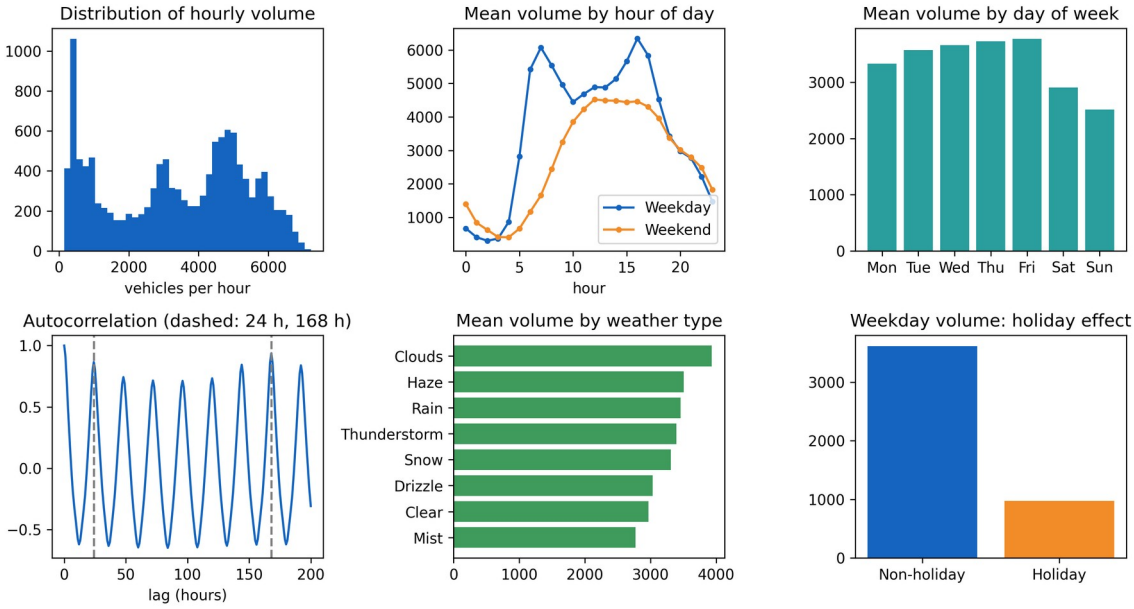


Figure 5.1: Exploratory Analysis of the Hourly Traffic Volume

5.2 Data Preprocessing Module

This module turns the raw records into a clean, continuous hourly series. Duplicate timestamps, which arise from several weather records for the same hour, are averaged, and the series is placed on a regular hourly grid. Gaps of up to three hours are filled by interpolation, but longer gaps are not, because values invented across a gap of days or months would make the forecasts look

better than they are. Instead, the longest unbroken segment, from 13 April 2017 to 30 September 2018, is kept. Weather type is one-hot encoded and holiday names are converted to a 0 or 1 flag. The order of the data is preserved throughout, because shuffling would let the models see the future. After the lag features are built, the modelling table contains 12,689 hourly records, which are split chronologically into 10,151 training hours and 2,538 test hours.

5.3 Feature Engineering Module

This module builds the inputs of the models, all from past information so that a forecast made at hour t uses only data available at hour t. The seven target lags of 1, 2, 3, 6, 12, 24 and 168 hours capture short-term momentum, the daily cycle and the weekly cycle. Two rolling means of the previous 3 and 6 hours smooth the noise, and the previous-hour values of 15 weather and holiday variables describe the conditions just before the forecast. Five calendar features, the hour, the day of the week, a weekend flag and the sine and cosine of the hour, describe the position in the daily and weekly cycle, and the sine and cosine pair lets 23:00 and 00:00 be treated as neighbours. Table 5.2 lists the 29 features.

Table 5.2: Feature groups

Group	Features	Count
Target lags	1, 2, 3, 6, 12, 24, 168 h	7
Rolling means	previous 3 h, previous 6 h	2
Past weather/holiday	previous-hour values	15
Calendar	hour, day, weekend, sin/cos hour	5

5.4 Forecasting Models Module

This module trains the forecasting models. Two baselines provide reference points: the naive model repeats the last value, and the seasonal naive model repeats the value 24 hours earlier. The time series model is a seasonal ARIMA with a 24-hour period [14]; three candidate orders were fitted on the last 3,000 training hours with statsmodels [13], and SARIMA$(2,0,1)\times(1,0,1)_{24}$ had the lowest AIC. Random Forest [12] averages many decision trees trained on bootstrap samples, which reduces variance and handles non-linear interactions, and XGBoost [11] builds trees one after another so that each corrects the errors of the previous ones. The hyperparameters of both ensembles were chosen by randomised search with six candidates and three-fold time series cross-validation on the training data only, using scikit-learn [10], and Table 5.3 gives the search space and the selected values.

Table 5.3: Hyperparameter search

Model	Parameter	Searched	Selected
Random Forest	trees	150, 300	150
Random Forest	max depth	none, 10, 20	20
Random Forest	min leaf size	1, 2, 5	2
Random Forest	features per split	sqrt, 0.5, 1.0	0.5
XGBoost	trees	200, 400, 600	600
XGBoost	max depth	3, 5, 7	5
XGBoost	learning rate	0.03, 0.05, 0.1	0.03
XGBoost	subsample	0.7, 1.0	0.7
XGBoost	column sample	0.7, 1.0	0.7

5.5 Evaluation and Analysis Module

This module judges the models and explains their behaviour. Accuracy is measured with the root mean squared error (RMSE), the mean absolute error (MAE), the coefficient of determination (R^2) and the mean absolute percentage error (MAPE), on one-hour-ahead forecasts. The main evaluation uses a chronological split in which the test hours lie after all training hours. A rolling-origin evaluation repeats the comparison over five expanding-window folds to check that the result does not depend on one lucky period. A feature ablation retrains the Random Forest with one group of features removed at a time, which shows what the model needs, and an error analysis breaks the errors down by hour of day, weekday and weekend, and rush hour and night, which shows where the model fails.

5.6 Signal Timing and Dashboard Module

This module converts the forecast into a green time and presents the results. Because the dataset has no signal timing, green time is derived by a linear rule: the forecast volume is scaled between the training minimum and the training 99th percentile into a share s between 0 and 1, and the green time is $G = G_{min} + s \times (G_{max} - G_{min})$, with $G_{min} = 10$ s and $G_{max} = 60$ s as design choices. The ideal green time is obtained by applying the same rule to the actual volume, and the fixed-time plan gives 30 seconds in every hour. The Streamlit dashboard has four tabs. The Forecast tab shows the headline indicators, the forecast curve and the model comparison, the Signal timing tab shows the ideal, forecast-based and fixed-time green times, the Insights tab shows the feature importances, the error by hour and the daily profile, and the Data tab lists the hourly records. The sidebar lets the user choose the model, the baselines to compare with, the window of the test period and the timing limits.

CHAPTER 6

IMPLEMENTATION

The project was implemented in Python in Google Colab notebooks and uses pandas and NumPy for data handling, scikit-learn [10] for Random Forest, cross-validation and metrics, XGBoost [11] for gradient boosting and statsmodels [13] for the seasonal ARIMA model. A single notebook runs the whole pipeline in order: it loads the dataset, removes duplicate timestamps, places the data on an hourly grid, keeps the longest unbroken segment, builds the features, splits the data chronologically, trains the baselines and the three models, and saves the results table, the figures and a summary of the dataset. A fixed random seed of 42 is used throughout, so the results can be reproduced.

A diagnostic step runs before modelling. It computes the autocorrelation at several lags, the correlation of each column with the target and the average volume by hour. This step is important because forecasting only works when the data contain a pattern. When the notebook was first run on an earlier smart mobility dataset, the diagnostic and the results showed that the volume behaved like random noise, with an R^2 close to zero for every model. The project therefore moved to the Metro Interstate dataset, which has strong daily and weekly structure, and this check is kept as a standard step of the notebook.

A second notebook cell produces the extra analyses reported in Chapter 7: the exploratory figure, the feature ablation, the rolling-origin evaluation and the error analysis. The ablation retrains the Random Forest with the tuned hyperparameters after removing each feature group, the rolling-origin evaluation reuses the tuned hyperparameters on five expanding-window folds,

and the error analysis groups the forecast errors of the Random Forest by hour and by day type.

The dashboard is a Streamlit application of about 200 lines. On start-up it repeats the preprocessing and feature construction, fits the Random Forest and XGBoost models on the training period and caches them, so it is ready within about a minute without any stored model files. Plotly is used for the interactive charts. The application is run with the command streamlit run traffic_dashboard.py, and its figures match the results of the notebook exactly.

CHAPTER 7

RESULTS AND DISCUSSION

This chapter reports the results of the forecasting models on the held-out test period of 2,538 hours. Table 7.1 compares the models. Random Forest achieved the best accuracy with an RMSE of 200.0 vehicles per hour, an MAE of 130.0, an R^2 of 0.989 and a MAPE of 5.1%, which is 74.9% lower RMSE than the last-value baseline and 46.7% lower than SARIMA. XGBoost was within 1.3% of Random Forest. The seasonal naive baseline has a lower MAE than the last-value baseline but a higher RMSE, because repeating yesterday fails on days that differ, such as weekends and holidays. SARIMA improves on both baselines but remains well behind the ensembles.

Table 7.1: One-hour-ahead forecasting performance on the test period

Model	RMSE	MAE	R^2	MAPE (%)
Naive (last value)	796.7	576.6	0.829	25.9
Seasonal naive (24 h)	964.5	504.5	0.749	21.8
SARIMA(2,0,1)×$(1,0,1)_{24}$	375.5	257.4	0.962	13.8
Random Forest	**200.0**	**130.0**	**0.989**	**5.1**
XGBoost	202.6	134.3	0.989	5.4

Figure 7.1 shows that the forecasts follow the daily peaks closely, and Figure 7.2 shows that the Random Forest forecasts lie along the diagonal over the whole range of volumes.

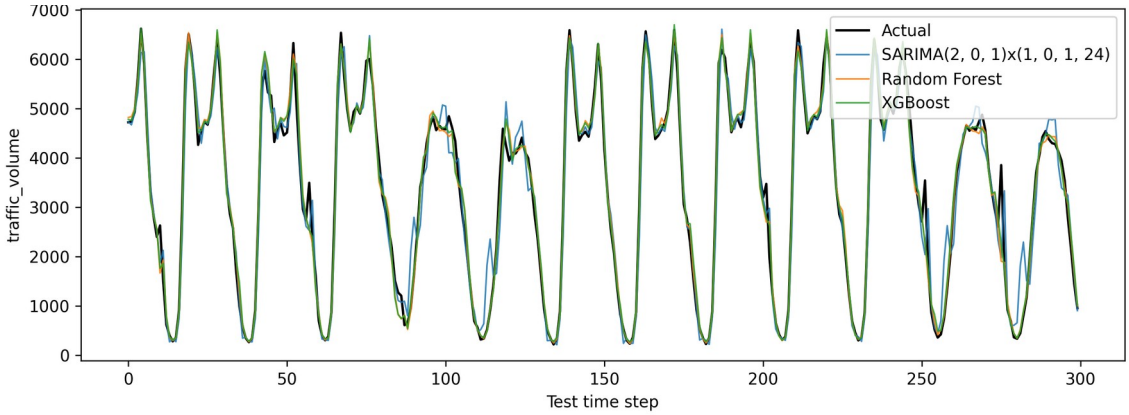


Figure 7.1: Actual and Forecast Traffic Volume on the Test Period

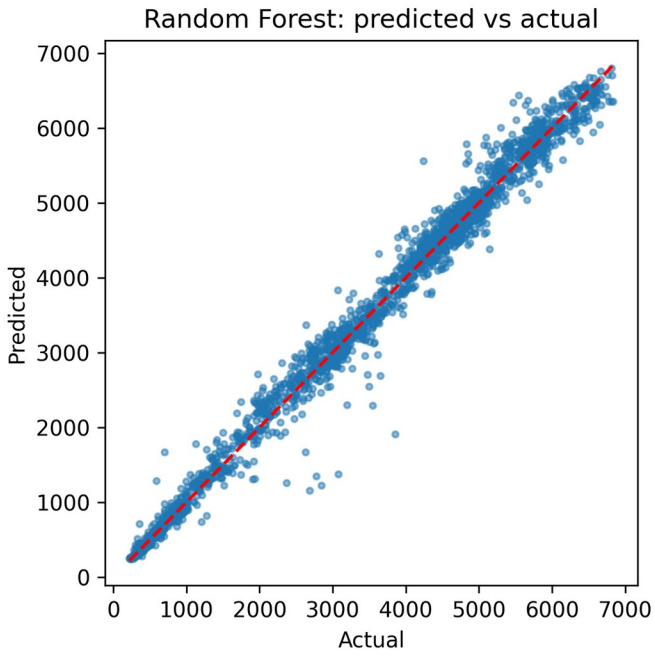


Figure 7.2: Random Forest Forecast against Actual Volume

A single split can be lucky, so the comparison was repeated over five rolling-origin folds (Table 7.2). Both ensembles stay far ahead of the baselines in every fold: the RMSE of Random Forest is between 58.6% and 75.5% lower than that of the naive baseline, and its mean RMSE is 262.9 against 819.6. The error of the ensembles varies more between periods (standard deviation about

55) than that of the baselines, and the hardest period is October 2017 to January 2018, which includes the winter holidays. XGBoost is slightly better than Random Forest in four of the five folds and on average (258.2 against 262.9), so the two ensembles are practically equivalent in accuracy, and the lower training cost of XGBoost makes it the more practical choice.

Table 7.2: RMSE over five rolling-origin folds

Fold	Test period	Naive	Seasonal naive	Random Forest	XGBoost
1	Jul–Oct 2017	848.9	982.6	244.9	239.8
2	Oct 2017–Jan 2018	810.9	1037.3	335.4	331.8
3	Jan–Apr 2018	809.8	1048.2	300.3	293.8
4	Apr–Jul 2018	830.1	1085.0	238.5	224.7
5	Jul–Sep 2018	798.1	943.7	195.4	201.1
Mean ± std		**819.6 ± 20.0**	**1019.4 ± 56.0**	**262.9 ± 55.1**	**258.2 ± 53.4**

Table 7.3 shows the feature ablation. Removing the weather and holiday features changes the RMSE by only 1.0, so weather adds very little once recent volumes are known. Removing the calendar features raises the RMSE by 4.8%, and the seven target lags alone give an RMSE of 211.5, only 5.7% worse than the full model, so most of the signal is the recent history of the series. The calendar features alone reach an R^2 of 0.959, which shows how regular the hourly and weekly pattern is, but their RMSE of 388.9 is almost twice that of the full model. Removing the weekly lag does not hurt at all (198.2), because the daily lag, the hour of day and the recent lags carry the same information.

Table 7.3: Random Forest feature ablation on the test period

Feature set	Features	RMSE	MAE	R^2
Full model	29	200.0	130.0	0.989
Without weekly lag (168 h)	28	198.2	129.3	0.989
Without daily and weekly lags	27	202.7	134.3	0.989
Without calendar features	24	209.6	137.8	0.988
Without past weather/holiday	14	201.0	131.4	0.989
Without rolling means	27	200.7	130.1	0.989
Target lags only	7	211.5	138.8	0.988
Calendar features only	5	388.9	211.1	0.959

Figure 7.3 shows the feature importances. For both ensembles the volume at the same hour one week earlier has the highest importance score, followed by the previous-hour volume and the hour of day. Importance scores are shared among correlated features and do not show that a feature is necessary, and the ablation shows that the model loses nothing without the weekly lag, so the ablation is the stronger evidence about what the model needs.

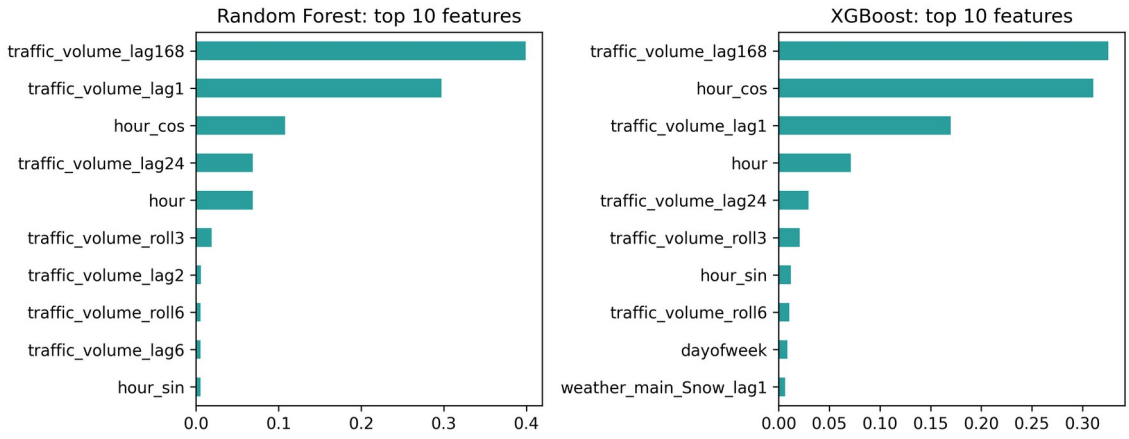


Figure 7.3: Top Features of Random Forest and XGBoost

Table 7.4 and Figure 7.4 break the errors of the Random Forest down. Weekday and weekend errors are almost identical (RMSE 200.7 and 198.1).

Errors are largest in the rush hours (240.5) and smallest at night (100.2). The hour with the largest RMSE is 23:00 (453) and the smallest is 03:00 (37). The mean error is +18 vehicles per hour, so the model slightly over-forecasts, and 91.0% of the forecasts are within 300 vehicles per hour of the actual value. The five largest errors all occur at 23:00, where the actual volume was 1,100 to 1,900 higher than forecast; these are late-evening surges that the earlier hours did not signal, and the data do not tell us their cause.

Table 7.4: Random Forest error by segment

Segment	Hours	RMSE	MAE
Weekdays	1,800	200.7	129.8
Weekends	738	198.1	130.5
Peak hours (07–08, 15–17 h)	530	240.5	180.4
Night (00–05 h)	630	100.2	55.2
Other hours	1,378	216.4	144.8

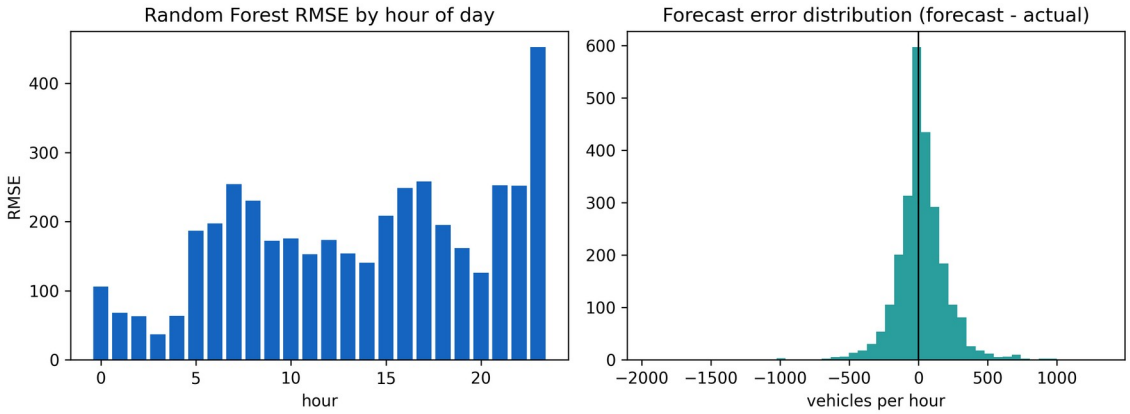


Figure 7.4: Random Forest Error by Hour and Error Distribution

Table 7.5 shows that XGBoost trained about twelve times faster than Random Forest, including the hyperparameter search, for an RMSE difference of only 2.6 vehicles per hour on the main split.

Table 7.5: Training time

Model	Time (s)
SARIMA (3 candidates)	25.3
Random Forest (6-candidate search)	112.0
XGBoost (6-candidate search)	9.5

Table 7.6 and Figure 7.5 show that the timing derived from the Random Forest forecast stays within about 1 second of the ideal green time on average, compared with 13.4 seconds for a fixed 30-second plan, which cannot follow the peaks and troughs of demand. This comparison measures how accurately the forecast can set green time. Because the ideal timing is computed from the same actual traffic by the same rule, it is not a measurement of reduced waiting time, which would need a traffic simulation or field data.

Table 7.6: Green time error against the ideal timing

Plan	Mean error (s)	RMSE (s)
Forecast-based (Random Forest)	0.99	1.52
Fixed-time (30 s)	13.43	15.21

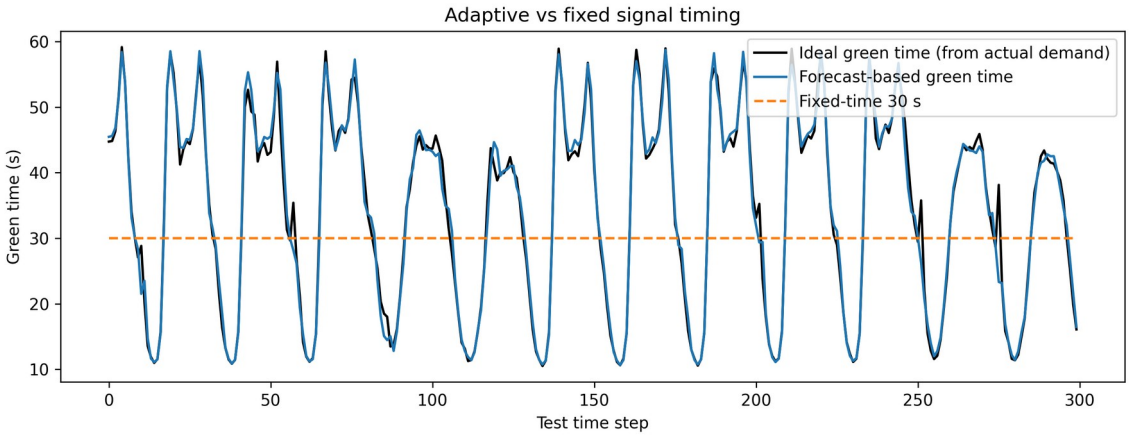
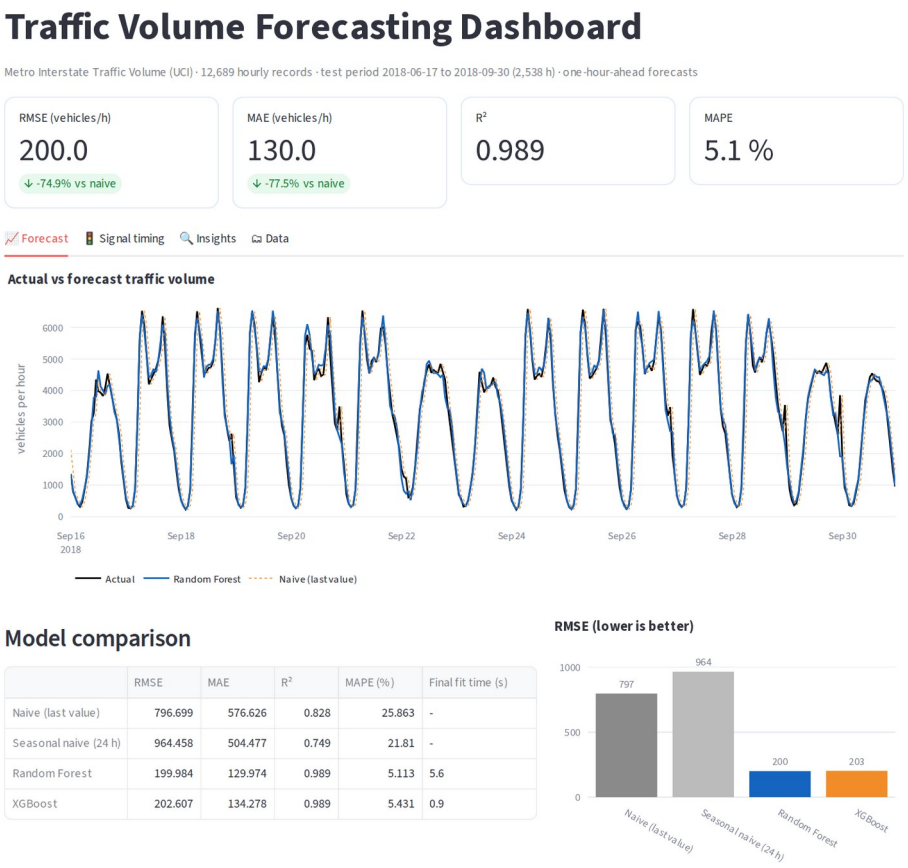


Figure 7.5: Forecast-Based, Ideal and Fixed-Time Green Time

Figures 7.6 to 7.8 show the dashboard, captured from the running application. The Forecast tab shows the headline indicators and the model comparison, the Signal timing tab shows the ideal, forecast-based and fixed-time green times, and the Insights tab shows the feature importances, the error by hour of day, the daily profile and the error distribution. The numbers in the dashboard match those of the tables above.



	RMSE	MAE	R^2	MAPE (%)	Final fit time (s)
Naive (last value)	796.699	576.626	0.828	25.863	-
Seasonal naive (24 h)	964.458	504.477	0.749	21.81	-
Random Forest	199.984	129.974	0.989	5.113	5.6
XGBoost	202.607	134.278	0.989	5.431	0.9

Figure 7.6: Dashboard, Forecast Tab

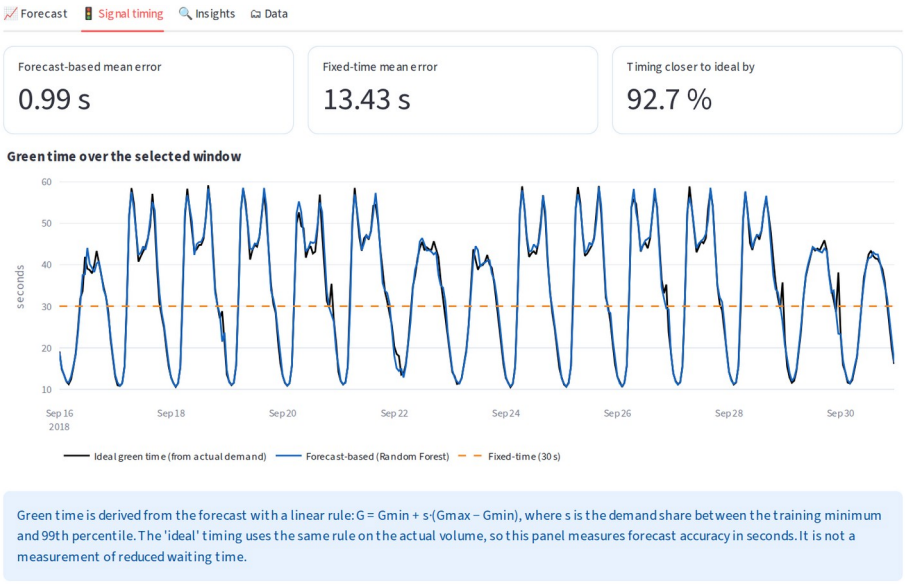


Figure 7.7: Dashboard, Signal Timing Tab

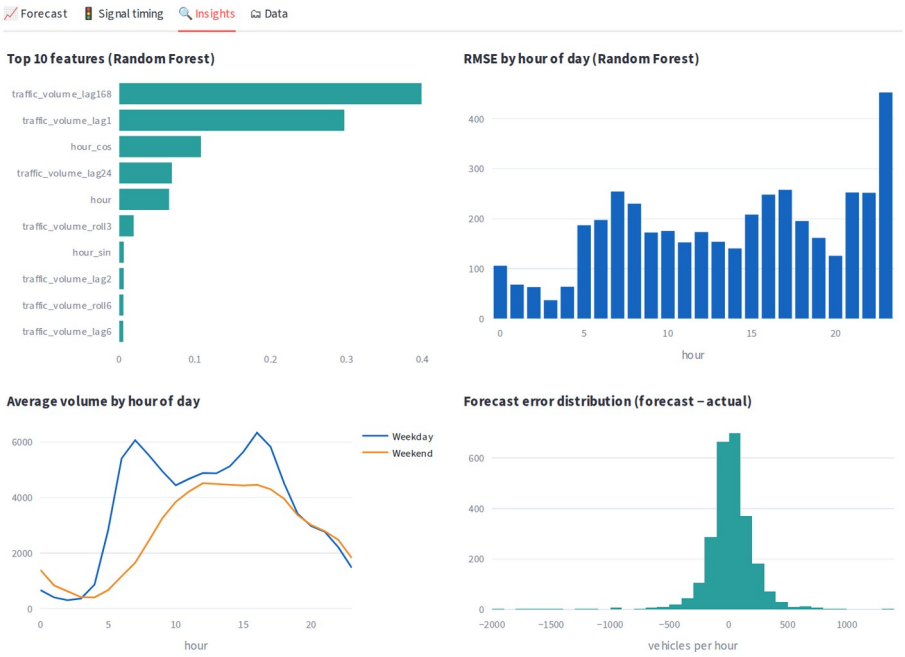


Figure 7.8: Dashboard, Insights Tab

In summary, tree-based ensembles with lagged volumes forecast hourly traffic far better than the baselines and SARIMA, most of their accuracy comes from the recent history of the series, weather adds little, and the largest errors occur

at 23:00 and in the rush hours. The limits of the study are that the data come from one highway location, the forecast horizon is one hour, the segment holds only 15 holiday hours, and the green time rule is an assumption that is not validated by a traffic simulation.

CHAPTER 8

FUTURE ENHANCEMENTS

While the current project forecasts hourly traffic volume at a single location one hour ahead, several extensions are possible. Forecasts several hours ahead would give more time to prepare signal plans, and evaluation on several intersections, at finer time resolution and on data that cover more holidays would show how well the findings transfer. The ablation suggests that a compact model with only seven lagged values is within 6% of the full model, so lightweight models could be deployed at many junctions.

The most important step is a traffic simulation that measures the waiting time and queue length obtained with the derived timings, since the present study measures forecast accuracy in seconds and not waiting time. The timing rule could also be improved with minimum green and clearance times, pedestrian phases and fairness constraints, and compared with reinforcement learning controllers. Hybrid forecasting models, for example a seasonal ARIMA model with a boosted residual model, and LSTM networks could be compared with the models of this project.

Finally, the dashboard could be connected to a live data feed so that forecasts update every hour, and it could be extended with alerts when the forecast error is unusually large. Coordination between neighbouring junctions and emergency-vehicle priority, which are studied in recent research, would turn the forecasting pipeline into a complete adaptive signal control system.

CHAPTER 9

REFERENCES

[1] H. Hajmohamed, D. Mahmud, L. Aldhaheri, S. Almentheri, S. Alqaydi, H. Sadia, and N. Saeed, “Deep Reinforcement Learning-Based Traffic Signal Control for Urban Congestion,” in *Proc. International Conference on Communication, Computing, Networking, and Control in Cyber-Physical Systems (CCNCPS)*, Dubai, UAE, Jun. 2025, pp. 37-42, doi: 10.1109/CCNCPS66785.2025.11135668.

[2] D. K. Kwesiga, A. Guin, and M. Hunter, “Adaptive Traffic Signal Control based on Multi-Agent Reinforcement Learning. Case Study on a simulated real-world corridor,” *arXiv preprint arXiv:2503.02189*, 2025. [Online]. Available: https://arxiv.org/abs/2503.02189

[3] W. Du, J. Li, and G. G. Wang, “PairUpLight: A Multi-agent Reinforcement Learning Approach for Coordinated Multi-intersection Traffic Signal Control,” in *Proc. 45th IEEE International Conference on Distributed Computing Systems (ICDCS)*, Glasgow, U.K., Jul. 2025, pp. 692-702, doi: 10.1109/ICDCS63083.2025.00073.

[4] A. Satheesh and K. Powell, “A Constrained Multi-Agent Reinforcement Learning Approach to Autonomous Traffic Signal Control,” *arXiv preprint arXiv:2503.23626*, 2025. [Online]. Available: https://arxiv.org/abs/2503.23626

[5] L. W. Ong, H. L. Khoo, K.-L. Yau, and C. F. Ong, “Multi-agent Reinforcement Learning for Traffic Signal Control: Comparison with Centralized, Decentralized and Hybrid Approaches,” in *Proc. 14th Asia-*

Pacific Conference on Transportation and the Environment (APTE 2025), Atlantis Press, 2025, pp. 50-60, doi: 10.2991/978-94-6463-972-8_6.

[6] E. Belt, L. Yao, X. Li, N. Wang, V. Shekhar, T. Koch, and E. Dugundji, "Forecasting traffic flow by vehicle category on a major highway impacted by road maintenance works," *Transportation Research Procedia*, pp. 1335-1352, 2025, doi: 10.1016/j.trpro.2024.12.128.

[7] F. Yuan, Y. Fan, X. Bing, Z. Tian, C. Yuan, and Y. Li, "Traffic flow forecasting, STL decomposition, hybrid model, LSTM, ARIMA, XGBoost, intelligent transportation systems," *arXiv preprint arXiv:2510.23668*, Oct. 2025. [Online]. Available: https://arxiv.org/abs/2510.23668

[8] I. D. Teledjieu and I. Shafique, "CityPulse: Real-Time Traffic Data Analytics and Congestion Prediction," *arXiv preprint arXiv:2506.01971*, 2025. [Online]. Available: https://arxiv.org/abs/2506.01971

[9] "Metro Interstate Traffic Volume," *UCI Machine Learning Repository*. [Online]. Available: https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume

[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785-794.

[12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, Oct. 2001.

[13] S. Seabold and J. Perktold, “statsmodels: Econometric and statistical modeling with Python,” in *Proc. 9th Python in Science Conference (SciPy)*, 2010, pp. 92-96.

[14] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[15] Streamlit Inc., “Streamlit Documentation.” Available: https://docs.streamlit.io